SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1005	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	Kanish Kumar V	**Reg. No.:**	192524361
Date:	10/10/2026	**Duration:**	60 minutes

Code of Conduct

I Kanish Kumar V (192524361) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Kanish Kumar V

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. **Understand the Problem Statement**
   - Explain the problem in your own words.
   - Identify the objectives of the proposed system.
   - State the scope of the problem.
2. **Identify Key Stakeholders and Challenges**
   - Identify all the stakeholders involved in the system.
   - Explain the roles and responsibilities of each stakeholder.
   - Discuss the major challenges and issues faced by the stakeholders.
3. **Analyze the Current Approach**
   - Describe the existing system or current process.
   - Identify its strengths and limitations.
   - Analyze the problems or gaps that need to be addressed.
4. **Propose a Suitable Solution Using Software Engineering Principles**
   - Design a software-based solution for the given problem.
   - Describe the major functional modules of the proposed system.
   - Identify the functional and non-functional requirements.
   - Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.
5. **Justify the Chosen Methodology/Framework**
   - Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).

- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

6. **Discuss Expected Outcomes and Limitations**
   - Describe the expected benefits of the proposed solution.
   - Explain how the solution addresses the identified challenges.
   - Discuss potential risks, implementation challenges, and limitations.
   - Suggest possible future enhancements.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/frame work and provides	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing;

	strong technical justification.			report lacks organization and clarity.

Criteria	**Mark**
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25

SMART LIBRARY SEAT RESERVATION AND STUDY SPACE MANAGEMENT SYSTEM

1. Understand the Problem Statement

1.1 Introduction

Libraries and learning centres provide students with a quiet environment for studying, reading, and preparing for examinations. However, during examination periods, the demand for study spaces increases significantly. Students often spend a considerable amount of time searching for vacant seats. Manual seat allocation and reservation methods can lead to confusion, double booking, and misuse of reserved seats.

The Smart Library Seat Reservation and Study Space Management System is a software-based solution designed to simplify the management of library study spaces. It allows students to check seat availability, reserve seats, check in when they arrive, and release seats after completing their study sessions.

The system also monitors occupancy, identifies unused reserved seats, sends notifications, and generates reports to help library administrators manage resources efficiently.

1.2 Objectives

The main objectives of the proposed system are:

1. To display real-time availability of library study spaces.
2. To allow students to reserve seats online.
3. To provide check-in and check-out facilities.
4. To monitor seat occupancy and reservation status.
5. To identify unused reservations and release seats automatically when appropriate.
6. To manage reservation durations and time limits.
7. To send reservation confirmations, reminders, and cancellation notifications.
8. To generate seat utilization reports for library administrators.
9. To reduce waiting time and improve student convenience.
10. To prevent double booking and reservation misuse.

1.3 Scope of the Problem

The system is intended for colleges, universities, public libraries, and learning centers that provide shared study spaces.

The scope includes:

- Student registration and login.
- Viewing available seats and study areas.
- Reserving seats for specified time slots.
- Checking in and checking out of reserved seats.

- Managing reservation cancellations and time limits.
- Detecting no-shows and releasing eligible reservations.
- Sending notifications to students.
- Providing an administrative dashboard.
- Generating occupancy and utilization reports.

The initial version may operate through a web application or mobile-friendly website. Integration with physical sensors, RFID cards, or QR codes can be introduced to improve occupancy monitoring in future versions.

The system does not initially cover unrelated library operations such as book lending, fines, catalog management, or purchasing library resources.

2. Identify Key Stakeholders and Challenges

2.1 Stakeholders and Their Responsibilities

1. Students

Students are the primary users of the system. They can view available study spaces, reserve seats, check in, cancel reservations, and release seats after use.

Challenges: Difficulty finding vacant seats, uncertainty about availability, missed reservations, and limited study space during examinations.

2. Librarians

Librarians supervise the library's study spaces and assist students with reservation-related issues. They can monitor occupancy, resolve booking conflicts, and manage seat availability.

Challenges: Manual monitoring, handling complaints, identifying unused seats, and managing large numbers of reservations.

3. Library Administrators

Administrators manage seat configurations, reservation policies, time limits, user permissions, and utilization reports.

Challenges: Efficiently allocating limited resources, preventing misuse, and making decisions based on accurate occupancy data.

4. System Administrator

The system administrator maintains the software, manages technical configurations, monitors system performance, and ensures data security and availability.

Challenges: Preventing unauthorized access, resolving technical problems, maintaining backups, and handling increased usage during peak periods.

5. College or Institutional Management

Institutional management evaluates library usage and decides whether additional seating, study areas, or infrastructure are required.

Challenges: Understanding demand patterns, allocating budgets, and planning future library facilities.

6. Notification Service Provider

An external email or messaging service may deliver reservation confirmations, reminders, and cancellation alerts.

Challenges: Delivery delays, service interruptions, and integration with the reservation system.

2.2 Major Challenges

The major challenges faced by stakeholders include:

- Limited seating capacity during examination periods.
- Students occupying seats without valid reservations.
- Students reserving seats but failing to arrive.
- Double booking caused by outdated availability information.
- Delays in identifying vacant seats.
- Difficulty monitoring actual occupancy.
- Unreliable network connectivity or notification delivery.
- Protecting student information from unauthorized access.

The proposed system addresses these challenges through centralized reservation management, automatic status updates, time-based reservation rules, and administrative monitoring.

3. Analyze the Current Approach

3.1 Existing System

In many libraries, seat allocation is handled manually. Students enter the library and search for available seats. In some cases, librarians maintain paper registers, spreadsheets, or physical reservation slips.

Students may need to ask library staff whether a seat is available. Reserved seats may remain unused because students fail to arrive, while other students continue searching for space.

Some libraries use basic online booking systems, but these may not provide automatic no-show detection, real-time occupancy monitoring, or detailed utilization reports.

3.2 Strengths of the Existing System

1. Simple to understand and operate.
2. Requires minimal technical infrastructure.
3. Can be inexpensive for small libraries.

4. Allows direct interaction between students and librarians.
5. Can continue operating without internet access when managed manually.

3.3 Limitations of the Existing System

1. Availability information may not be updated immediately.
2. Students waste time searching for vacant seats.
3. Manual records are prone to errors.
4. Multiple students may attempt to occupy or reserve the same seat.
5. Unused reserved seats may remain unavailable to other students.
6. Monitoring occupancy requires considerable staff effort.
7. Reservation history and utilization data are difficult to analyze.
8. Manual processes become inefficient when the number of students increases.

3.4 Problems and Gaps to Be Addressed

The existing approach lacks a centralized platform that combines reservation, occupancy monitoring, check-in, check-out, and reporting.

The main gaps are the absence of reliable real-time availability, automatic release of unused reservations, consistent reservation rules, and data-driven planning tools.

The proposed system will address these gaps by maintaining a centralized database and updating seat status whenever a reservation or occupancy event occurs.

4. Propose a Suitable Solution Using Software Engineering Principles

4.1 Proposed System

The proposed Smart Library Seat Reservation and Study Space Management System is a centralized software application that manages the complete seat reservation lifecycle.

Students log in to the application and view available seats. They select a seat and reserve it for an allowed time slot. On arrival, they check in using a reservation code or QR code. After studying, they check out so that the seat becomes available to other students.

If a student does not check in within the permitted grace period, the system marks the reservation as a no-show and releases the seat according to library policy. The system also provides administrators with occupancy dashboards and utilization reports.

For accurate physical occupancy monitoring, the system can initially rely on check-in and check-out records. Optional sensors or access-control devices can later be integrated to detect actual presence more reliably.

4.2 Major Functional Modules

1. User Management Module

- Provides student registration and login.
- Authenticates users.
- Supports role-based access for students, librarians, and administrators.
- Allows authorized users to update their profiles.

2. Seat Availability Module

- Displays available, reserved, occupied, and unavailable seats.
- Organizes seats by floor, zone, or study area.
- Updates availability when reservation statuses change.
- Prevents unavailable seats from being booked.

3. Reservation Management Module

- Allows students to select a seat and time slot.
- Confirms valid reservations.
- Prevents overlapping bookings for the same seat.
- Supports cancellation and reservation history.
- Enforces booking duration and advance-booking rules.

4. Check-in and Check-out Module

- Verifies reservations when students arrive.
- Records check-in and check-out times.
- Supports QR-code-based check-in if implemented.
- Updates seat status after a student checks out.

5. Occupancy Monitoring Module

- Tracks reservation and occupancy statuses.
- Identifies seats that are available, occupied, or reserved.
- Records occupancy changes.
- Can integrate with sensors for automatic physical occupancy detection.

6. No-Show Detection Module

- Identifies reservations that have not been checked in within the allowed grace period.
- Marks qualifying reservations as no-shows.
- Releases the associated seat when policy permits.
- Records no-show events for administrative review.

7. Notification Module

- Sends reservation confirmation messages.
- Provides reminders before the reservation begins or expires.

- Notifies students of cancellations and automatic releases.
- Alerts students when reservation-related changes occur.

8. Reports and Analytics Module

- Generates daily, weekly, and monthly utilization reports.
- Calculates seat occupancy rates.
- Identifies peak study hours.
- Summarizes cancellations and no-show rates.
- Helps administrators plan future seating capacity.

9. Administration Module

- Allows administrators to add, modify, or deactivate seats.
- Configures reservation duration and grace periods.
- Manages study-area availability and maintenance periods.
- Provides dashboards for monitoring system activity.

4.3 Functional Requirements

Functional requirements describe the operations the system must perform.

1. The system shall allow students to register and log in.
2. The system shall display seat availability by study area.
3. The system shall allow students to reserve available seats.
4. The system shall prevent conflicting reservations for the same seat and time slot.
5. The system shall allow students to check in and check out.
6. The system shall update reservation and seat statuses.
7. The system shall detect reservations that exceed the allowed check-in grace period.
8. The system shall release seats according to configured reservation rules.
9. The system shall send reservation notifications.
10. The system shall generate seat utilization reports.
11. The system shall allow authorized administrators to manage seats and reservation policies.
12. The system shall maintain reservation history for auditing and analysis.

4.4 Non-Functional Requirements

Non-functional requirements describe the quality attributes of the system.

1. Performance: Seat availability and reservation operations should respond quickly, even during examination periods.

2. Reliability: The system should process reservations correctly and recover from temporary failures without losing confirmed bookings.

3. Security: Passwords must be stored securely, communications should use HTTPS, and access must be restricted according to user roles.

4. Usability: The interface should be simple, mobile-friendly, and understandable to students with different levels of technical knowledge.

5. Scalability: The application should support additional users, seats, study areas, and library branches as demand increases.

6. Maintainability: The software should use modular code, meaningful naming, documentation, and automated tests to simplify future modifications.

7. Availability: The service should remain accessible during library operating hours, with suitable backup and recovery procedures.

8. Consistency: Concurrent reservation attempts must not result in double booking.

9. Privacy: The system should collect only necessary student information and restrict access to personal and reservation records.

10. Compatibility: The application should work with commonly used web browsers and mobile devices.

4.5 System Architecture

The system can follow a three-tier architecture.

Presentation Layer: The web or mobile-friendly interface used by students, librarians, and administrators.

Application Layer: The backend services that implement authentication, reservation rules, check-in, occupancy monitoring, notifications, and reporting.

Data Layer: The database that stores user accounts, seat information, reservations, occupancy events, and audit records.

The layers communicate through defined interfaces, making the application easier to test, maintain, and extend.

4.6 Basic System Workflow

1. The student logs in to the application.
2. The student views the available seats.
3. The student selects a seat and an available time slot.
4. The system verifies availability and confirms the reservation.
5. The system sends a confirmation notification.
6. The student checks in within the allowed period.
7. The system updates the reservation and occupancy status.
8. The student checks out after completing the study session.
9. The system releases the seat for the next student.
10. If the student fails to check in, the system identifies the no-show and releases the seat according to configured rules.

11. Administrators review utilization reports and improve resource allocation.

4.7 Application of Software Engineering Principles

Modularity: Separate modules manage users, seats, reservations, notifications, and reports. Changes to one module can be made with minimal impact on others.

Scalability: A centralized database, efficient queries, and independently manageable application services allow the system to grow as the number of students and seats increases.

Maintainability: Coding standards, documentation, automated tests, and separation of responsibilities make debugging and enhancements easier.

Reliability: Database transactions, validation, backups, error handling, and recovery procedures help maintain accurate reservation records.

Usability: Clear seat-status indicators, simple booking steps, accessible controls, and timely notifications improve the student experience.

Security: Secure authentication, role-based authorization, encrypted communication, input validation, and audit logging help protect the system and its users.

Data Integrity: Database constraints and transaction-based booking operations prevent conflicting reservations and inconsistent seat statuses.

5. Justify the Chosen Methodology/Framework

5.1 Selected Methodology: Agile with Scrum

Agile with Scrum is selected as the software development methodology because the proposed system contains several interconnected modules and may require changes based on feedback from students, librarians, and administrators.

Instead of developing the entire application in one long phase, the team can build and test the system incrementally through short development cycles called sprints.

5.2 Why Agile with Scrum Is Suitable

1. Changing Requirements: Reservation policies, time limits, and check-in methods may change after stakeholders evaluate the application. Agile supports these changes.

2. Early Delivery: Basic functions such as login, seat availability, and reservation can be delivered before advanced reporting and sensor integration.

3. Continuous Feedback: Students and librarians can test each increment and suggest improvements.

4. Early Problem Detection: Regular testing helps identify double-booking errors, interface problems, and performance issues before the system becomes too complex.

5. Improved Collaboration: Developers, testers, and stakeholders can review progress regularly.

6. Flexible Prioritization: Important functions can be developed first, while optional features can be scheduled for later sprints.

5.3 Proposed Sprint Plan

Sprint 1: Planning and User Management

- Gather requirements.
- Identify stakeholders.
- Design the database and user interface.
- Implement login and role-based access.

Sprint 2: Seat Availability and Reservations

- Create the seat management module.
- Display seat availability.
- Implement booking and cancellation.
- Prevent conflicting reservations.

Sprint 3: Check-in and Occupancy Monitoring

- Implement check-in and check-out.
- Update seat status.
- Introduce no-show detection and automatic release rules.

Sprint 4: Notifications and Reports

- Add reservation confirmations and reminders.
- Develop utilization reports.
- Create the administrative dashboard.

Sprint 5: Testing and Deployment

- Perform integration, security, usability, and performance testing.
- Fix defects.
- Deploy the application.
- Collect feedback and plan improvements.

5.4 Support for Maintenance

After deployment, Agile supports continued improvement through feedback, defect prioritization, and future sprints. New features such as sensor integration, predictive occupancy analysis, and multi-library support can be introduced without redesigning the entire application.

6. Discuss Expected Outcomes and Limitations

6.1 Expected Benefits

1. Better Seat Utilization: Unused reservations can be identified and released, allowing more students to use available seats.

2. Reduced Search Time: Students can view seat availability before entering or moving around the library.

3. Improved Convenience: Online booking, simple check-in, and notifications make reservation management easier.

4. Reduced Reservation Misuse: Time limits, check-in requirements, and no-show rules discourage students from blocking seats unnecessarily.

5. Better Administrative Control: Librarians can monitor study areas and manage reservation policies through a central dashboard.

6. Data-Driven Planning: Reports reveal peak hours, popular study areas, and patterns of unused capacity.

7. Improved Transparency: Students receive clear information about seat status and reservation details.

6.2 How the Solution Addresses Existing Challenges

- **Problem:** Students waste time searching for seats.
  Solution: Display current seat availability through the application.
- **Problem:** Reserved seats remain unused.
  Solution: Detect no-shows and release seats after the configured grace period.
- **Problem:** Multiple students attempt to book the same seat.
  Solution: Use database transactions and reservation constraints.
- **Problem:** Manual monitoring increases staff workload.
  Solution: Automate status updates, reminders, and reporting.
- **Problem:** Administrators lack accurate utilization information.
  Solution: Generate reports based on reservation and occupancy records.
- **Problem:** Students forget their reservation times.
  Solution: Send confirmations and timely reminders.

6.3 Potential Risks and Implementation Challenges

1. Incorrect Occupancy Information: A check-in record may not always indicate that a student is physically occupying a seat. Optional sensors or periodic verification can improve accuracy.

2. Network Failures: Students may be unable to access the system during connectivity problems. Backup procedures and suitable recovery mechanisms are required.

3. Concurrent Booking Conflicts: Multiple students may attempt to reserve the same seat simultaneously. Atomic database transactions and appropriate constraints are necessary.

4. Privacy and Security Risks: Personal data or reservation history may be exposed if access controls are weak. Security testing and data protection practices are essential.

5. User Adoption: Some students and librarians may prefer manual methods. Training, clear instructions, and a simple interface can encourage adoption.

6. Implementation Cost: Software development, hosting, maintenance, and optional sensors require financial resources.

7. Notification Failures: Email or messaging services may experience delays. The application should retain reservation information internally and provide an in-app notification option.

8. Incorrect No-Show Releases: Students may arrive late or experience check-in difficulties. A configurable grace period and clear release policy can reduce unfair cancellations.

6.4 Limitations of the Proposed System

1. Accurate physical occupancy detection may require additional hardware.
2. The system depends on reliable power, network access, and backend services.
3. Reservation rules must be configured appropriately for different libraries.
4. Initial implementation requires time, staff training, and financial investment.
5. Utilization reports are only as accurate as the recorded reservation and occupancy data.
6. Automatic seat release cannot completely prevent misuse if students bypass the check-in process.

6.5 Future Enhancements

The system can be improved in the following ways:

1. **IoT Sensor Integration:** Install occupancy sensors to detect whether seats are physically occupied.
2. **QR Code Check-in:** Allow students to scan a code placed near their reserved seat.
3. **Mobile Application:** Develop dedicated Android and iOS applications.
4. **AI-Based Demand Prediction:** Analyze historical usage to predict busy periods and support better seating plans.
5. **Smart Notifications:** Notify students when preferred seats become available.
6. **Multi-Library Support:** Extend the system to manage seats across multiple libraries or campuses.
7. **Accessibility Features:** Provide filters for accessible seating and other special requirements.
8. **Digital Student ID Integration:** Support authorized institutional identity systems for faster check-in.
9. **Advanced Analytics:** Compare utilization across different floors, study zones, and time periods.
10. **Offline Recovery Support:** Introduce controlled fallback procedures to handle temporary network outages.

Conclusion

The Smart Library Seat Reservation and Study Space Management System provides an effective software-based solution to the problems associated with manual library seat management. It enables students to check availability, reserve seats, check in, and release seats conveniently while helping librarians monitor occupancy and manage resources efficiently.

The proposed system applies software engineering principles such as modularity, scalability, maintainability, reliability, usability, security, and data integrity. Agile with Scrum is suitable because it enables incremental development, regular testing, and continuous stakeholder feedback.

Although challenges such as network failures, implementation costs, and accurate occupancy detection must be addressed, the system can significantly improve the utilization of limited study spaces. Future enhancements involving sensors, mobile applications, and predictive analytics can further strengthen its effectiveness and support better library planning.